

# SQL

Structured query language

DB relazionale è la relazione che intercorre tra le righe e colonne della tabella

T SQL = "t" Transact

<> è l'operatore diverso che si usa in sql

DDL = Data definition language

DML = Data modification language



COOLING TOWERS  
since 1981

SQL = STRUCTURED QUERY LANGUAGE  
DB RELAZIONALE È LA RELAZIONE CHE INTERCORRE  
TRA LE RIGHE E COLONNE DELLA TABELLA

T SQL = "t" TRANSACT

SQL È UNO STANDARD ISO E ANSI  
NTR MATH  
TAG ETAS  
ENA TINI  
RDN RODI  
AAI INATU  
TIA CARU  
ION ALDE  
N N

<> OPERATORE DIVERSO

SQL FAREMO 2 OPERAZIONI: DDL E DML

DDL: DATA DEFINITION LANGUAGE

DML: DATA MODIFICATION LANGUAGE ES. INSERT UPDATE DELETE

| DML      | OPERAZIONI  | DDL    |
|----------|-------------|--------|
| CREATE   | CREARE      | INSERT |
| DROP     | DISTRUGGERE | DELETE |
| ALTER    | MODIFICARE  | UPDATE |
| TRUNCATE |             | SELECT |

SUVOLO TABELLA



ILMED IMPIANTI SRL  
VIALE DEI MARESCHI 15 - 10051 AVIGLIANA (TO) - ITALY  
TEL +39 011.932.55.55 - FAX +39 011.936.72.89



sql è uno standard iso e ansi  
con la delete scegli quali dati eliminare invece la truncate elimina tutto

Non è case sensitive

## FUNZIONI DI BASE

**ctrl n** new file

**ctrl u** seleziona nome db

**f5** execute oppure **ctrl e**

**ctrl shift r** fa il refresh

Lo star non si usa quasi mai perchè se noi facciamo uno star che prende tutti i campi della tabella se noi programmatori andiamo a modificare la tabella con un alter table aggiungendo un campo poi la select con lo star è probabile che si spacchi e quindi non funziona più la query.

Select filtro in verticale, invece la where fa il filtro in orizzontale

### ALIAS

**SELECT nome AS NomeHobbit, cognome AS CognomeHobbit FROM Hobbits**

con AS(alias) cambiamo la visualizzazione del nome del campo della colonna, prima era solo nome e cognome, ora invece è NomeHobbit e CognomeHobbit.

l'alias si può usare nell'order by ma non nella where

per inserire gli spazi nella rinominazione della tabella bisogna inserirlo tra le parentesi quadre

**SELECT nome AS NomeHobbit, cognome AS [Cognome Hobbit] FROM Hobbits;**

### Truncate/Delete

Quando facciamo la **delete** guarda tutti i record e gli controlla con la condizione della where se è soddisfatta, invece la **truncate** prende l'intero contenuto della tabella e ne scarta.

## Ordinamento ASC/DESC

ASC ascendente come ordinamento

DESC discendente come ordinamento

## Concat

se dobbiamo unire le colonne della tabella possiamo fare nome + ' ' + cognome però dobbiamo dargli il nome con l'alias AS

oppure con la concat **SELECT CONCAT (Nome,' ', Cognome) as NomeCognome**

LA DIFFERENZA TRA I 2 È CHE CON IL + NON TE LO FA FARE CON TIPI DI DATI DIVERSI, INVECE CONCAT CONCATENA ANCHE TIPI DIVERSI ES STRINGHE CON INT.

con il + se hai un null e hai una stringa+null ti da null invece con concat ti da il valore stringa

**USARE LA CONCAT AL POSTO CHE IL + È FORTEMENTE CONSIGLIATO**

## TOP

**SELECT top (4) Nome, Cognome**

**FROM Hobbits**

**Order By Nome;**

con il top selezioniamo i primi 4 della tabella, si mette order by perchè sennò prende sempre i primi 4 però sono messi in ordine casuale quindi non sappiamo bene cosa prende

**SELECT TOP (4) Hobbits.Nome, Hobbits.Cognome**

**FROM Hobbits**

**ORDER BY Nome desc;**

si può sempre omettere lo schemas quando lo schema è dbo quindi non scrivere dbo.Hobbits ma solo Hobbits.

**Però è sempre meglio specificare lo schemas quindi mettere dbo.Hobbits**

quando scriviamo davanti il nome del database.qualcosa riusciamo ad accedere a quel qualcosa di quel database anche se non abbiamo selezionato quel determinato database

SERVER → DATABASE → SCHEMA → TABELLE → COLONNE → DATI

Per creare lo schema

**go**

**CREATE SCHEMA lotr**

**go**

**ALTER SCHEMA lotr TRANSFER dbo.hobbits;**

La select distinct serve a togliere i duplicati però nella select distinct devo indicare **per forza** il campo della order by, invece senza distinct posso omettere il campo della order by

INSERIRE FOTO SCHEMA A CASA DA TELEFONO

**SELECT DISTINCT Cognome as surname**

**From lotr.Hobbits**

**where cognome <= 'Brandybuck'**

**order by surname asc;**

Se non c'è la riga di order by in questo caso fa comunque un order by implicito perchè trova l'operatore <=

BETWEEN per valori compresi

## **NULL**

Si comporta in modo anomalo perchè quando lo confronto con gli operatori es > < <> ecc... da sempre false, invece bisogna usare **IS NULL, IS NOT NULL.**

Per NULL bisogna usare per forza IS NULL per trovare quelli null oppure is NOT NULL

l'operatore diverso non tira fuori i null

```
SELECT Cognome, nome  
FROM Iotr.Hobbits  
WHERE Cognome <> 'baggins'  
ORDER BY Nome;
```

invece così si:

```
SELECT Cognome, nome  
FROM Iotr.Hobbits  
WHERE Cognome <> 'baggins' OR Cognome IS NULL  
ORDER BY Nome;
```

## LIKE

```
WHERE Cognome like 'B%'
```

Vede tutti i cognomi che iniziano per B

```
WHERE Cognome like 'Ba%'
```

Vede tutti i cognomi che iniziano per Ba

```
WHERE Cognome like '%s'
```

vede tutti i cognomi che finiscono per s

```
WHERE Cognome like '%s%'
```

vede tutti i cognomi che contengono la s

```
WHERE COGNOME LIKE '%n_'
```

vede che la penultima è una n e dopo c'è ancora un carattere

```
WHERE COGNOME LIKE '%[aeiou]%'
```

vede se c'è un a e i o u all'interno, è come se fosse un or, non un and

## CASE...WHEN

verifica la condizione, se viene soddisfatta passa al prossimo sennò succede qualcosa

```
CASE WHEN condizione1 THEN conseguenza1  
WHEN condizione2 THEN conseguenza2
```

...

```
ELSE conseguenza n  
END
```

```
select Nome,  
Cognome,  
case when Cognome = 'Baggins' then 'buono'  
when Nome = 'sam' then 'buonissimo'  
when Nome = 'gollum' then 'cattivo'  
else 'buonino'  
END AS buonezza  
from lotr.Hobbits
```

### CTE - common table expression

```
with CTE_Hobbits AS (  
select Nome,  
Cognome,  
case when Cognome = 'Baggins' then 'buono'  
when Nome = 'sam' then 'buonissimo'  
when Nome = 'gollum' then 'cattivo'  
else 'buonino'  
END AS buonezza  
from lotr.Hobbits)  
select nome, cognome, buonezza  
from CTE_hobbits Where buonezza = 'buonino';
```

è come se creassi una tabella che poi posso interrogare dopo.  
nella cte devo per forza dare il nome a tutte le tabella, qui l'alias AS buonezza è  
fondamentale perchè sennò non funziona niente

Bisogna sempre mettere il ; prima della whit

### IFF

```
select Nome,  
Cognome,  
IFF (Cognome = 'baggins',1,0) as isbaggins  
from lotr.Hobbits  
se è cognome = baggins è uguale a 1 sennò 0
```

## Count/group by

conta i valori della tabella

**SELECT COUNT(\*) FROM lotr.Hobbits**

**SELECT COUNT(1) FROM lotr.Hobbits**

**è uguale mettere \* o 1**

se il valore è nullo non lo conta, es:

**SELECT COUNT(cognome) FROM lotr.Hobbits**

stamperà 6 al posto di 7 perchè c'è un cognome null

**SELECT COUNT(ISNULL(Cognome, '-')) from lotr.Hobbits**

stamperà 7 perchè sostituisce il null

**SELECT cognome, COUNT(cognome) FROM lotr.Hobbits**

**group by cognome**

ma se facessi

**SELECT nome, COUNT(cognome) FROM lotr.Hobbits**

**group by cognome**

non funziona perchè la group by è su cognome e non su nome

**SELECT nome, COUNT(cognome) FROM lotr.Hobbits**

**group by nome**

controlla che ogni nome ha un cognome

**quando si fa una group by nella select o devono essere scritti nella group by o devono essere aggregati(ovvero inseriti in una funzione di raggruppamento es max, min, count ecc...), non esiste che selezionate un campo che non è presente nella group by.**

l'ordine è SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY per gli umani

il pc invece fa: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY

ABBIAMO AGGIUNTO UN CAMPO NULL

ORA QUANTI NULL STAMPANO QUESTE 2 OPERAZIONI?

**select cognome, COUNT(COGNOME) AS CONTEGGIO**

**from lotr.Hobbits**

**GROUP BY COGNOME**

QUESTO 0

```
select cognome, COUNT(1) AS CONTEGGIO
from lotr.Hobbits
GROUP BY COGNOME
QUESTO 2
```

```
SELECT COGNOME, COUNT(1) AS CONTEGGIO
FROM LOTR.Hobbits
GROUP BY COGNOME
WHERE COGNOME IN ('TOOK', 'BAGGINS')
ORDER BY COGNOME DESC;
```

La where la usiamo quando dobbiamo filtrare i risultati, la having la utilizziamo solo insieme alle funzioni di raggruppamento.

## TIPI DI DATI

Numeri:

- interi: int, bigint, smallint, tinyint
- decimali: numeric(m,n), decimal(m,n) m numeri totali della cifra, n sono dopo la ,

Booleani: bit (0,1, null)

Caratteri: char(n), varchar(n), nchar(n), nvarchar(n). char la lunghezza è fissa, varchar variabile tu metti la grandezza massima e il restante non viene riempita

char e varchar sono ASCII invece nvarchar e nchar sono UNICODE

Date: date(yyyy:mm:dd), time(hh:mm:ss:mmm), datetime(yyyy:mm:dd hh:mm:ss:mmm),

datetime2(yyyy:mm:dd hh:mm:ss:mmm), smalldatetime (hh:mm:ss)

datetime va di 3 millisecondi in 3 millisecondi invece datetime2 è più preciso.



## **FUNZIONI DI CONVERSIONE**

Cast(val as tipoDiDato)

TRY\_CAST(val as tipoDiDato)

CONVERT(tipoDiDato, val, cod)

## **ISTRUZIONI DDL**

CREATE TABLE TABELLA(

col1 tipoDiDato {{not}null} (se non scrivo nulla la colonna è nullable) {default (")} (il default con il not null sostituisce una stringa null con " in questo caso),

...

)

DROP TABLE TABELLA (distrugge tabella)

ALTER TABLE

## **ISTRUZIONI DML**

INSERT INTO tabella (col1, col2, ...)

VALUES (val1, val2, ...)

INSERT INTO tabella (col1, col2, ...)

SELECT col1, col2, ...

FROM tabella2

UPDATE schema.tabella

set col1 = val1,

col2 = val2...

WHERE cond

```
DELETE  
FROM schema.table  
WHERE cond1 = pippo
```

## TRANSACTION

visto che usiamo t sql possiamo fare un rollback scrivendo prima dell'operazione però begin transaction per andare a recuperare i dati persi o sovrascritti. bisogna stare attenti ai **lock** perchè quando facciamo una **begin transaction** è come se mettessimo un lock sulla tabella e non possiamo accederci o altri sviluppatori non possono accedervi da altre query finchè non chiudiamo l'operazione con **COMMIT** o facciamo **ROLLBACK**.

**select \* from lotr.Hobbits WITH (nolock)** con questo comando anche se c'è un lock sulla tabella da parte di un utente può vedere i dati prima del rollback. quindi lui ha visto una situazione che non esisterà perchè se poi faccio il rollback per tornare indietro i dati che ha visto l'altro utente facendo questo comando non saranno veritieri.

Le caratteristiche di una buona transazione sono:

**ACID: atomicità, consistenza, isolamento, durabilità**

per atomicità significa o va tutto a buon fine oppure niente, non è ammesso il caso ibrido in cui la prima parte va bene e la seconda no.

quando noi facciamo un update è autocommittante, anche senza vederlo fa una begin transaction e la chiude quando ha finito.

sarebbe sempre meglio mettere in una tabella di log tutte le operazioni che stiamo facendo.

INSERT INTO LOG VALUES 'APRO OPERAZ BONIFICO'

CONSISTENZA significa che ciò voglio scrivere venga effettivamente scritto

ISOLAMENTO: livelli di isolamento

DURABILITÀ significa ciò che faccio rimanga nel tempo

## LIVELLI DI ISOLAMENTO

read committed

read uncommitted vede tutto ciò che sta succedendo anche se l'utente 1 ha messo un lock

Il nolock serve per aggirare il livello read committed.

con serializable non posso ne leggerli ne modificarli

snapshot vedo una "foto" dei dati al momento in cui inizia una transazione

Le vecchie versione dei dati vengono salvate nel tempdb nell snapshot e read committed

### 1 READ UNCOMMITTED

✓ Vedi anche dati non ancora confermati (dirty read)

✗ Il più pericoloso

### 2 READ COMMITTED

✓ Vedi solo dati confermati

✗ Se rileggi, i dati possono cambiare (non repeatable read)

### 4 SERIALIZABLE

✓ Massima sicurezza

✓ Le transazioni vengono eseguite come se fossero in sequenza

✗ Più lento (blocchi più forti)

| Isolation level    | Supported | Honored        |
|--------------------|-----------|----------------|
| READ UNCOMMITTED   | No        | Not guaranteed |
| READ COMMITTED     | Yes       | Yes            |
| REPEATABLE READ    | No        | No             |
| SNAPSHOT ISOLATION | No        | No             |
| SERIALIZABLE       | No        | No             |

## **CONSTRAINT**

```
ALTER TABLE [Iotr] [Hobbits] ADD CONSTRAINT [DF_Hobbits_Eta] DEFAULT ((0))  
FOR [Eta] GO
```

con la constrain costringiamo a mettere delle condizioni nella colonna ad esempio se vogliamo fare la colonna che sia per forza maggiore di 0 possiamo mettere una constraint > 0.

se troviamo un valore null viene messo false .

## **NOZIONI IMPORTANTI**

**QUANDO DEVI ESPRIMERE UNA CONDIZIONE SU UN'OPERAZIONE DI RAGGRUPPAMENTO USA SEMPRE HAVING, SE INVECE VOGLIO FILTRARE I RISULTATI AL DI FUORI DI UNA FUNZIONE DI RAGGRUPPAMENTO ALLORA USA LA WHERE**

## **RIASSUNTO GIORNATA 2/15 OTTOBRE**

SQL fa prima la distinct poi la top

es: SELECT DISTINCT TOP(N)

col1 AS alias, .....

from schema.tabella AS aliastable

where cond (=, <>, <, >, LIKE, NOT LIKE, IN, NOT IN, BETWEEN, or, and, is null, is not null)

GROUP BY col1, col2

METTI FOTO DA TELEFONO CON WHERE CON GLI OR E I WHERE  
CON GLI IN

## **Chiave primaria**

È una o più colonne che identificano unicamente un record nella tabella.  
quali caratteristiche ha? scrivilo

## **Chiave surrogata**

È quando non riesco a trovare una chiave nelle colonne della tabella creo una  
colonna a parte con una chiave primaria

## JOIN

**full join** Se voglio i risultati di tutte le tabelle

**left join** tutti gli elementi della prima tabella

**right join** tutti gli elementi della seconda tabella

**inner join** solo gli elementi dell'intersezione delle tabelle

**cross join** fa il prodotto cartesiano degli elementi utile per ad esempio battaglia navale

```
from schema.tab1 t1
{full/left/right/inner} join
schema.tabs2 t2 on t1.colx = t2.coly
```

```
select l.Titolo AS titoloLibro, l.AnnoDiUscita as annoDiUscitaLibro, f.AnnoDiUscita as
annoDiUscitaFilm
from film f inner join libri l
ON f.titolo = l.titolo
```

```
select
isnull(l.titolo, f.titolo) as provatitolo,
l.AnnoDiUscita as annoDiUscitaLibro,
f.AnnoDiUscita as annoDiUscitaFilm
from film f full join libri l
ON f.titolo = l.titolo
```

```
select l.titolo as titoloLibro ,
l.AnnoDiUscita as annoDiUscitaLibro,
f.AnnoDiUscita as annoDiUscitaFilm ,
isnull(cast(f.AnnoDiUscita as varchar(10)), '-') as annoDiUscitaFilm,
case when f.Titolo = l.Titolo then cast(f.AnnoDiUscita as varchar(10)) else '-' end
from libri l left join film f
ON f.titolo = l.titolo
```

**COSA IMPORTANTE NEL PRIMO CASE ANCHE SE NON ENTRA NEL THEN  
DEL CASE WHEN STAMPA COMUNQUE IL TRATTINO PERCHÈ IL CAST È  
SEMPRE IMPLICITO**

## UNION / UNION ALL

quando abbiamo le tabelle con lo stesso tipo e numero di colonne possiamo fare la union.

**la union fa implicitamente una distinct invece la union all non fa la distinct.**

se ci fosse un order by va in quella sotto? cerca ma credo di sì

quando facciamo join è come se facessimo un confronto tra le due tabelle invece con la union è proprio come se mettessero le tabelle una sotto l'altra.

```
select annodiuscita, count(1) from film  
group by AnnoDiUscita  
union  
select annodiuscita, count(1) from libri  
group by AnnoDiUscita
```

**select f.\***

stampa tutti i campi della tabella f, se non mettesti f e facessi una join stamperebbe i campi di tutte le tabelle nella join

## DISEGNO DEL DATABASE

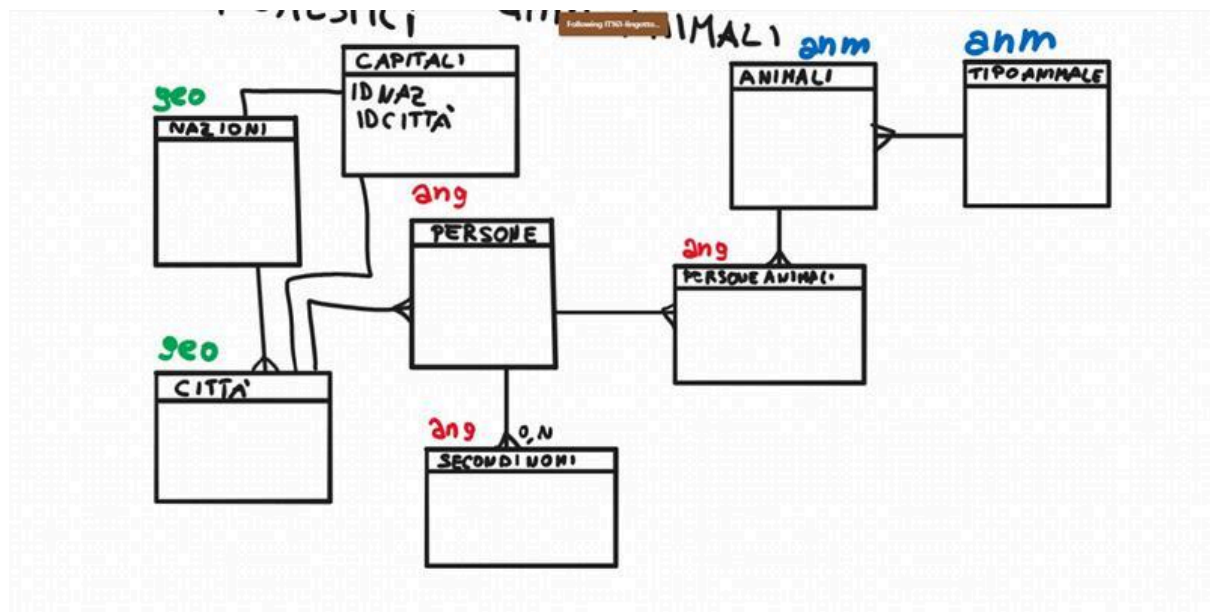
per esprimere la relazione molti a molti ci serve una terza tabella

relazione gerarchica: nella tabella figlia ci devono essere 2 entità che si riferiscono a una stessa es. tabPersone IDa - tabGerarchiaIDb, IDa\_1, IDa\_2

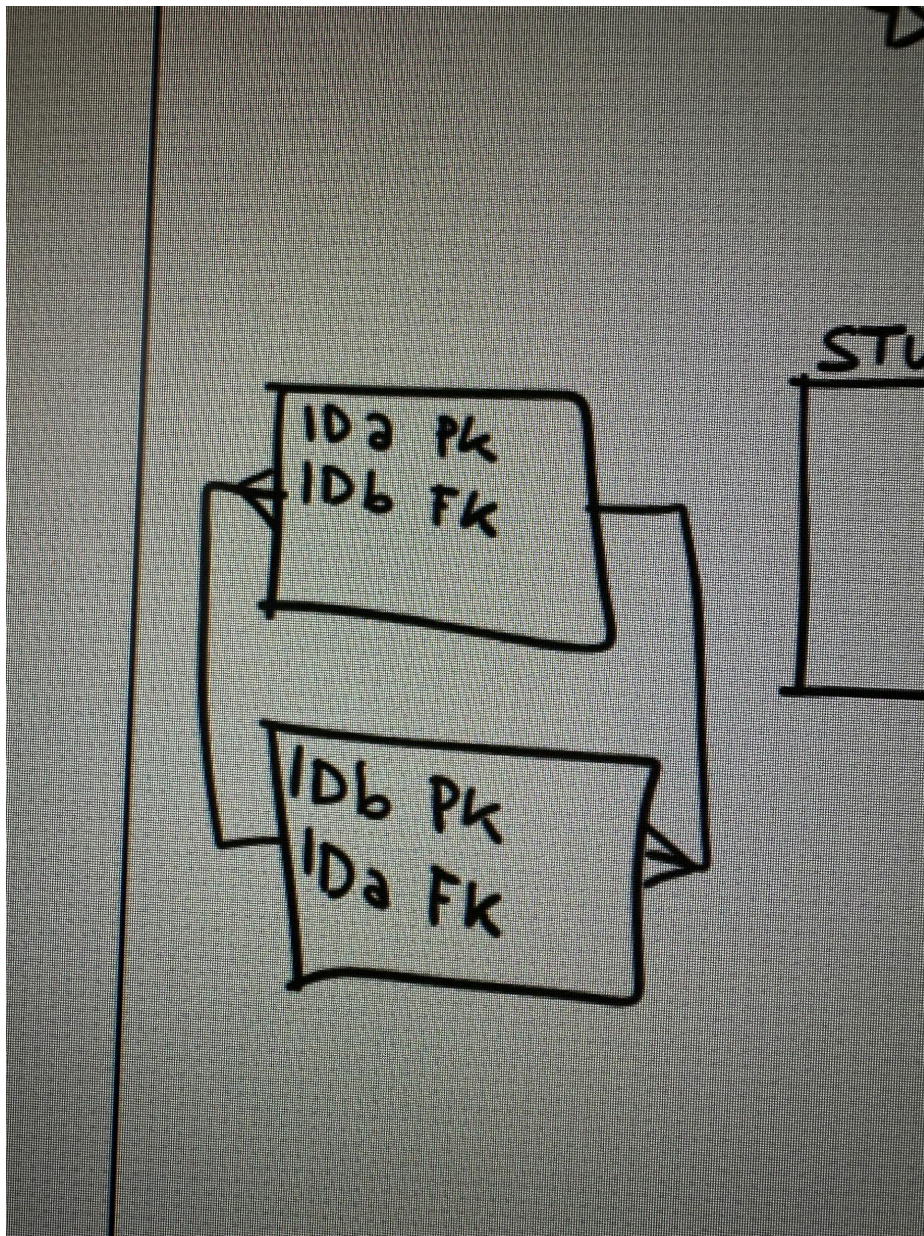
**CHIEDERE DOMANI LA TABELLA DI GERARCHIA**

la foreign key è meglio se non è nullable però non è detto che non sia nullable.

non posso avere una foreign key di libri di un numero che non ha corrispondenza sulla tabella madre.







Questo non si fa perché non si ha la garanzia che sia non null perché prima o poi diventa null

## COLLATION

per gestire le stringhe in case sensitive

**COLLATE** keyword per rendere case sensitive

## INTEGRITÀ REFERENZIALE

Se devi eliminare un campo dalla tabella padre devi prima eliminare tutti i figli

## SUBQUERIES

```
SELECT *  
FROM tab1  
WHERE col1 = (select che ritorna una riga e una colonna) es. una top(1) o max ecc..
```

```
SELECT *  
FROM tab1  
WHERE col1 IN (select che ritorna una riga e una colonna)
```

```
SELECT *  
FROM (select che ritorna n righe x n colonne)
```

```
select *  
from ang.persone  
where year(DataDiNascita) = (select max(year(datadinascita)) as annorecente  
from ang.Persone)
```

```
SELECT Nome, Cognome  
from ang.persone  
order by cognome  
offset 2 row  
fetch next 4 rows only;
```

NON LO CHIEDE ma serve a prendere tot righe a partire da

keyword DECLARE

quando un insieme può avere elementi che non ci sono nell'altro facciamo una full join senno' basta la left join

```
select p.cognome, p.nome, string_agg(s.secondonome, ', ' )  
from ang.Persone p  
inner join ang.SecondiNomi s  
on s.Personald = p.Personald  
where p.Nome = 'giovanni'  
group by p.Cognome, p.Nome
```

per mettere nella stessa colonna secondinomi altri secondi nomi